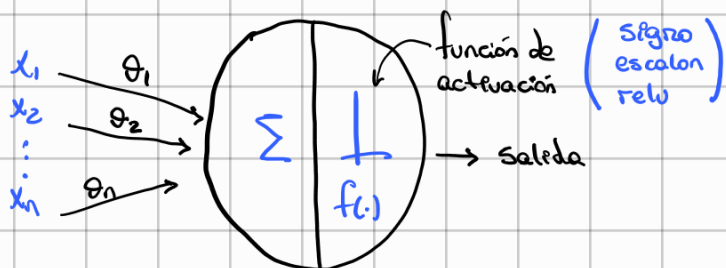
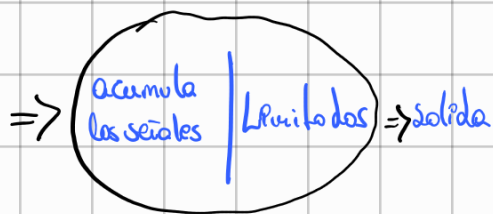


Abstracción de las Neuronas

Entradas:

- Sinapsis de otras neuronas
- olor, dolor, etc



Representación de Perceptron

Perceptron: la primera máquina de aprendizaje. Introducido por Frank Rosenblatt en 1958 es el modelo más sencillo de una red neuronal artificial y la unidad básica de inferencia que opera como un discriminador lineal.

Componente Biológico Perceptron Función

- Dendritas	- Entradas (x, x_0)	- Recibe la información datos del exterior
- Sinapsis	- Pesos (θ, θ_i)	- Determina la fuerza o la importancia de la conexión. Son multiplicadores numéricos que ponderan la entrada.
- Soma/Núcleo	- Unión Sumadora (Σ) y el sesgo (β)	- Acumula la señal ponderada. El sesgo ajusta el intercepto o el umbral.
- Principio todo o nada	- Función de activación (f)	- Decide si el perceptron se excita (se activa) para producir una salida binaria.
- Plasticidad	- Ajuste de los pesos (θ, β)	- El aprendizaje o "inteligencia" se manifiesta en la modificación iterativa de los pesos.

Funcionamiento del Perceptron

Se divide 2 fases principales durante la propagación hacia adelante

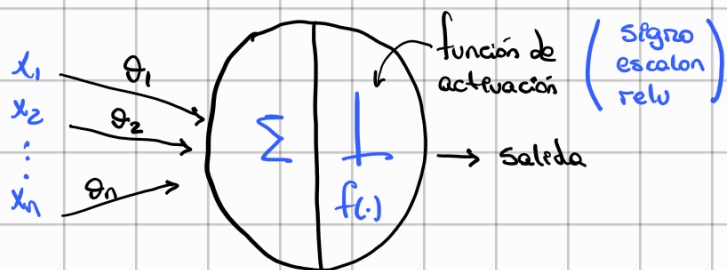
Paso 1: El cálculo del Potencial Sináptico (Suma Ponderada). Simplemente multiplicar cada entrada (x_i) por su peso sináptico asociado (θ_i) y sumar todos estos productos.

$$\text{Suma}(I) = \sum_{i=1}^n \theta_i x_i + b$$

$$x_0 = 1$$

$$\theta_0 = b$$

$$\sum_{i=1}^{n+1} \theta_i x_i$$



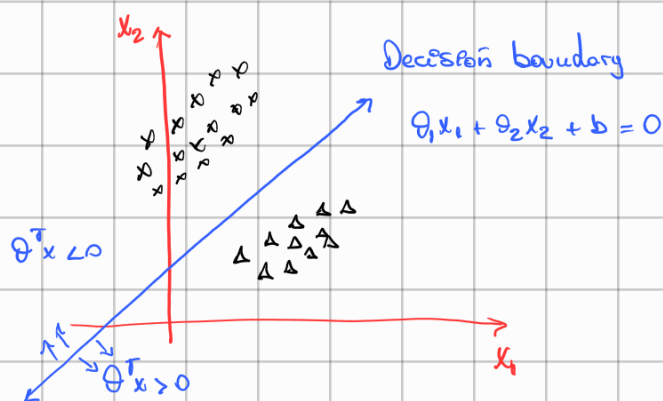
Paso 2: Aplicación de la función activación (Decisión del umbral). El resultado del potencial sináptico se pasa a la función de activación. En el perceptron esta función suele ser la función sigmo (un gran levitador) o la función escalón la cual produce una salida binaria.

$$\text{Salida}(y) = f(\text{suma}) = \begin{cases} 1, & \text{si } \sum \theta_i x_i \geq 0 \\ -1, & \text{si } \sum \theta_i x_i < 0 \end{cases}$$

es esto una lpuca?

→ lo correcto es decir un hiperplano.

La conclusión de esto es que el perceptron puede separar 2 clases.



Tarea Demuestra matemáticamente que el perceptron puede separar 2 clases si son linealmente separables.



Es un problema linealmente no separable. No podemos reducir el error cerca de cero. No importa los ajustes que hagamos en los sinapsis.

El problema es más difícil que la capacidad del modelo solucionador.

"en algunos casos esto también puede ser underfitting"

Ajuste correcto de pesos.

$x(n)$ es correctamente clasificado por $\theta(n)$

$$\theta(n+1) = \theta(n)$$

Si $\theta^T x(n) \geq 0$, $x(n) \in C_{+}$

si no lo clasifica

$$\theta(n+1) = \theta(n) - \eta x(n)$$

Tarea de aprendizaje
 $\eta > 0$

$$\theta_{n+1} := \theta_n + \alpha (y^{(n)} - h_{\theta}(x^{(n)})) x^{(n)}$$

$$x^{(n)} \in \mathbb{R} \quad y^{(n)} \in \{0, 1\}$$

hipotesis

$$h_{\theta}(x) = g(\theta^T x)$$

la probabilidad de
 q nos de el modelo

→ suma ponderada
 f. activación

$$g(z) = \begin{cases} 0, & z \geq 0 \\ 1, & z < 0 \end{cases}$$

$\theta :=$ inicialización (θ) Para $e = 1, \dots, n$

$$\theta := \theta + \alpha (y^{(e)} - h_{\theta}(x^{(e)})) x^{(e)}$$

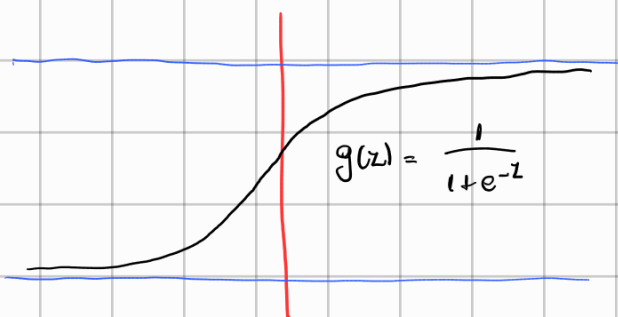
Regresión logística

$$x^{(e)} \in \mathbb{R}^n \quad y^{(e)} \in \{0, 1\}$$

hipotesis

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}} \quad \text{función logística}$$



$$\lim_{z \rightarrow -\infty} g(z) = 0$$

$$\lim_{z \rightarrow +\infty} g(z) = 1$$

$$P(y^{(e)} = 1 | x^{(e)}; \theta) = h_{\theta}(x)$$

$$P(y^{(e)} = 0 | x^{(e)}; \theta) = 1 - h_{\theta}(x)$$

En general $P(Y | X, \theta) = [h_{\theta}(x)]^y [1 - h_{\theta}(x)]^{1-y}$

Distribución de Bernoulli

$$L(\theta) = \prod_{i=1}^M P(y^{(i)} | x^{(i)}; \theta)$$

$$\begin{aligned} \log(L(\theta)) &= l(\theta) \\ &= \sum_{i=1}^M y^{(i)} \log h_{\theta}(x^{(i)} + (1-y^{(i)})) \log(1 - h_{\theta}(x^{(i)})) \end{aligned}$$

$$\frac{\partial g(z)}{\partial z} = g(z)(1-g(z))$$

Para una muestra

$$l(\theta) = y \log g(\theta^T x) + (1-y) \log(1 - g(\theta^T x))$$

$$\nabla_{\theta} l(\theta) = y \cdot \frac{1}{g(\theta^T x)} g'(\theta^T x) x + (1-y) \cdot \frac{1}{(1-g(\theta^T x))} (-1) g'(\theta^T x)$$

$$= y \frac{1}{g(\theta^T x)} g(\theta^T x)(1-g(\theta^T x)) x - (1-y) \frac{1}{(1-g(\theta^T x))} g(\theta^T x)(1-g(\theta^T x)) x$$

$$= y(1-g(\theta^T x))x - (1-y)g(\theta^T x)x$$

$$= [y(1-g(\theta^T x)) - (1-y)g(\theta^T x)]x$$

$$= [y - yg(\theta^T x) - g(\theta^T x) + yg(\theta^T x)]x$$

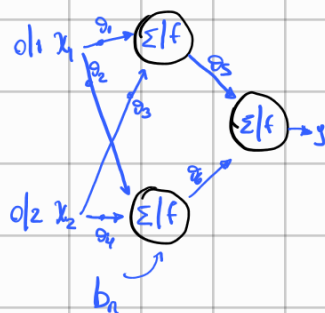
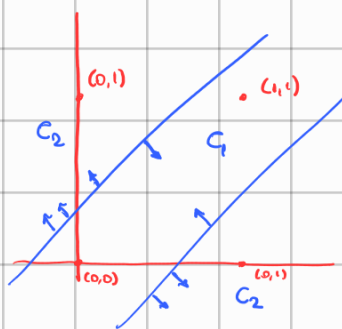
$$= [y - g(\theta^T x)]x$$

$$= [y - h_{\theta}(x)]x$$

$$\theta_{n+1} = \theta_n + \alpha []$$

Tarea: Utiliza el perceptron para aprender la función lógica "or" con entradas bipolares.

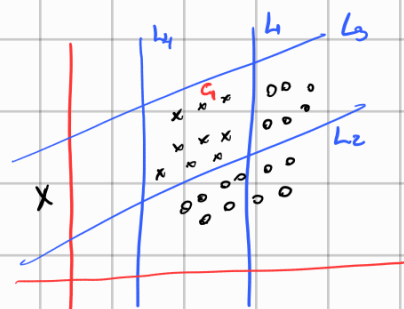
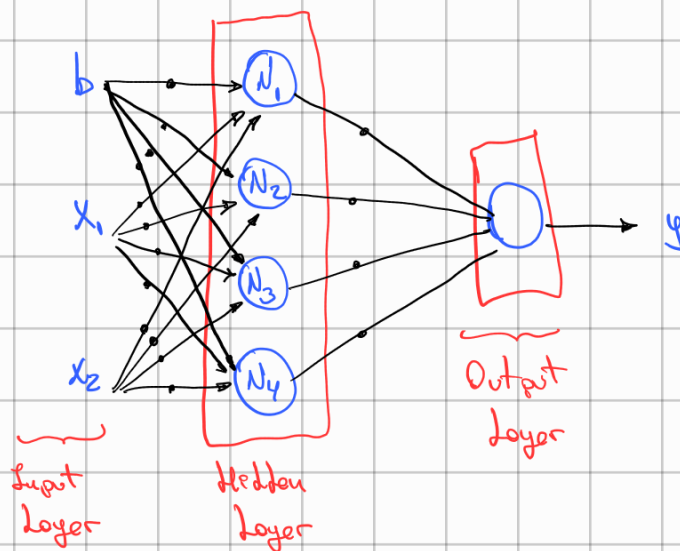
Casos	Entradas (x_1, x_2)	Salida deseada (z)
e_1	$(-1, -1)$	-1
e_2	$(1, -1)$	-1
e_3	$(1, 1)$	1
e_4	$(-1, 1)$	1



No se puede resolver con una sola línea (perceptron)

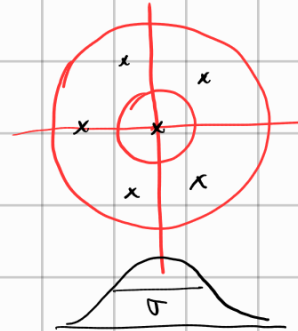
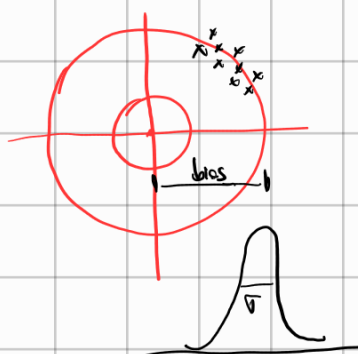
Esta representación nos permite separar los no lineales (con 2 líneas). Usando la función signo por simplicidad.

Para 1 capa oculta



Alto sesgo (Underfitting)

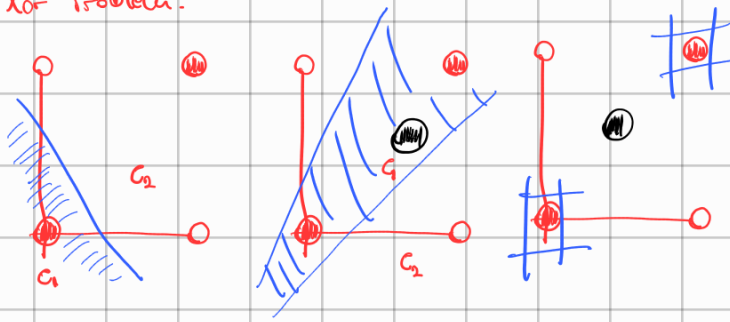
Alta varianza (Overfitting)



El modelo es muy seguro de sí mismo (poca despersión) pero está consistentemente equivocado

Hay mucha incertidumbre

Xor Problem.



• Para 1 neurona no se resuelve el problema

• Para 2 neuronas se resuelve el problema

• Resuelve el problema con mayor certeza (accuracy)

¿es necesario?

No, es una solución memorizada

Learning algorithms para MLP (Multi Layer Perceptron)

también conocido como "retroalimentación" MLP

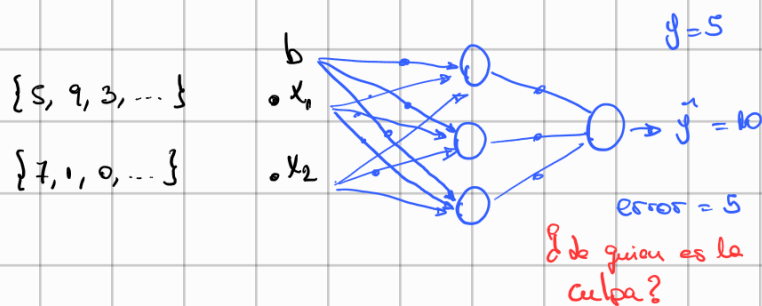
Feed Forward (F.F)

¿D' significa f.f?

→ la información / cálculos fluyen como un río

¿Cuál es el aprendizaje?

Necesitamos resolver el problema asignado "credit assignment problem"



- El error debe ser cercano a cero

- Como corregimos a partir del error

- Necesitamos algo que pueda aprovechar la forma del error y nos ayude a ajustar los pesos

¿Como propagamos el error hacia atrás en la red para poder corregirlo?

Resp: back propagation.

Precisión / baja varianza

Exactitud / bajo sesgo